



Feature additions and integration spec

Everything the finished production app still needs, drawn from the backend review, the UI/UX system and the v8 POC, with the exact API contracts, business rules, file-by-file changes and tests needed to fit into the existing backend without breaking it.

17

backend items (B1–B17)

14

frontend items (F1–F14)

6

decisions to confirm first

4

delivery phases

Built against: the existing backend in `D:\Metalix inventory\Backend` (Node 22 · Express 5 · Mongoose 8 · Zod 3 · Decimal128 · JWT/Argon2)

Sources compared: Metal Scrap Backend & Database Production Guide · Metalix UI/UX System v1.0 · metal-scrap-management-poc-v8 (source purchase company)

What is inside

Read chapters 1–3 before writing any code: they fix the rules and the decisions. Chapters 4–6 are the backend work, one spec card per item. Chapters 7–8 are the frontend work. Chapters 9–10 cover testing, rollout and how every POC feature maps to an item.

PLAN

1 · Summary and delivery phases	3
2 · Rules every addition must follow	4
3 · Decisions to confirm first (D1–D6)	6
Master list of all 31 items	7

BACKEND: PHASE A (P0)

B1 · Delivery capacity endpoint	9
B2 · Totals for the filtered list	10
B3 · Multi-status filter and status summary	11
B4 · Server-side sorting	12
B5 · Last purchase rate	12

BACKEND: PHASE B (P1)

B6 · Wider search	13
B7 · Materials with stock position	13
B8 · Dashboard extensions	14
B9 · Company statistics	14
B10 · Activity feed	15
B11 · Bought vs delivered trend	15
B12 · Rate analysis and rate trend	16
B13 · Migration 002	16

BACKEND: PHASES C AND D (P2)

B14 · CSV export	17
B15 · Demo / seed data script	17
B16 · Refresh token in httpOnly cookie	18
B17 · Idempotency keys on create	18

FRONTEND

7 · Screen changes F1–F8	20
Rate analysis page (wireframe)	23
8 · API client F10–F14, error dictionary	24

QUALITY AND ROLLOUT

9 · Test plan and definition of done	27
10 · Rollout, env vars, README changes	29
Appendix · POC feature → item map	31

Page numbers are approximate; use the item IDs (B1, F3, D2...) to cross-reference. They are stable and are also meant as ticket names.

Where the project stands and what is left

The backend implements every endpoint in the production guide and passes 111 tests. What is missing are the endpoints the new frontend needs and the POC features that neither the backend nor the UI/UX doc carried over.

Three reviews, one list

REVIEW	FINDING	ITEMS
Backend vs production guide	All 20 endpoints present and correctly guarded. Only <code>tests/business-rules.test.js</code> was renamed (its cases live in <code>tests/unit/utls.test.js</code>). Nothing to add.	none
Backend vs UI/UX system	Every backend API is used by a screen. The UI needs 5 endpoints it lists on its own page 30 plus 9 gaps found by comparing mockups with the code.	B1–B5, B11, B12, B14, B16, B17, F9, F10
POC v8 vs backend + UI/UX	15 POC features are missing or only partly planned: rate analysis, company-wise figures, material position everywhere, activity feed, wider search, and more.	B6–B10, B12, B15, F2–F8

Delivery phases

Each phase ends with `npm run check` green (lint and all tests), an updated README and a short report, the same way the first six phases were done.

A · Safe daily work P0 - B1 capacity - B2 list totals - B3 multi-status - B4 sorting - B5 last rate - F1, F2, F9, F10, F13	B · POC parity P1 - B6 search - B7 material position - B8 dashboard - B9 company stats - B10 activity - B11 trend, B12 rates - B13 migration - F3–F8	C · Convenience P2 - B14 CSV export - B15 seed script - F12 export button	D · Hardening P2 - B16 cookie session - B17 idempotency - F14 client support
---	---	--	---

What does not change

- No breaking changes.** Every existing endpoint keeps its path, request and response. New data is added as new fields, new optional query parameters or new endpoints. The current 111 tests must keep passing unmodified.
- Business rules stay where they are.** The three delivery limits, ledger replay, PO invariants and rate snapshot are not duplicated. New read endpoints call the same service functions the write paths use.
- Stored data is unchanged.** No existing collection gets a new required field. Migration 002 only adds indexes and one new collection.

Effort at a glance

Phase A ≈ 4–5 dev-days backend + 5 frontend. Phase B ≈ 6 backend + 6 frontend. Phases C and D ≈ 4 backend + 2 frontend. Estimates assume one developer who knows this codebase and include tests.

Rules every addition must follow

These are the conventions the existing backend already uses. New code that breaks one of them is a bug, even if its own tests pass.

1 · Layering

Route → `validate(schema)` → controller → service → repository.
 Controllers stay thin and only call `ok()` / `created()` from `utils/apiResponse.js`. Business decisions live in services. Queries live in repositories.

2 · Response envelope

Success: `{ success: true, data, meta? }`. Error:

`{ success: false, error: { code, message, details?, requestId } }` through `AppError`. List meta always comes from `pageMeta()`; new list extras go inside `meta` (for example `meta.summary`).

3 · Exact decimals

Quantities (3 dp), rates (2 dp) and money (2 dp) are `Decimal128` in MongoDB, `Decimal.js` in code and **strings** in JSON, using `QTY_SCALE`, `RATE_SCALE` and `MONEY_SCALE`. Never `Number()` a quantity. Weighted averages are `value ÷ quantity`, as in `avgRate()`.

4 · ObjectIds in aggregations

Mongoose casts ids in `find()` but **not** in `aggregate()`. Every `$match` built from query input must use `toObjectId()`. This was a known bug in the guide's reference code; B2, B6, B9 and B10 all hit it.

5 · Activity vs position

Activity (bought, sold, values, averages) uses the period `[from, to]` on business dates. Position (stock, PO remaining, supplier pools, buy needed) is the balance **as of to**. Never mix the two in one number. Default `to` is `businessToday(BUSINESS_TIMEZONE)`, not UTC.

6 · Business dates

Dates are `YYYY-MM-DD` strings validated by `businessDate` and stored at UTC midnight. "As of a day" includes that whole day (`endOfDayUTC`). Technical time (`createdAt`) is only for ordering activity, never for accounting.

7 · Roles

All `GET` routes are open to OWNER and VIEWER. Every write (`POST` / `PATCH`) uses `authorize(ROLES.OWNER)`. Read-only helper endpoints (capacity, last rate) are `GET` and open to both roles.

8 · Route order

Fixed sub-paths (`/capacity`, `/summary`, `/last-rate`, `/export.csv`) must be registered **before** `/:id`. Otherwise `idParams` rejects them with 422 `VALIDATION_ERROR`.

9 · Writes

Any write that affects stock runs in `withTransaction`, locks POs/materials through `LockableRepository.lock()` in sorted order, and replays the affected ledgers. None of the items here adds a new stock write; B17 wraps existing writes without changing them.

10 · Validation

Every query parameter is declared in Zod, with unknown keys rejected as today. Booleans use `queryBoolean`, never `z.coerce.boolean` (it turns `"false"` into true). Enumerations use `z.enum`. Lists (comma-separated) are parsed and each value validated.

Definition of done for every B-item

- Zod schema, route, controller, service and repository code in the existing files named in the spec card.
- Integration tests in `tests/integration/` using the real replica set (`mongodb-memory-server`): happy path, validation, VIEWER access, and one edge case per rule.
- README API table and "deviations" section updated; `npm run check` green.

Decisions to confirm before coding

The POC, the UI/UX doc and the backend disagree in six places. Each has a recommendation; confirm or change it, and record the answer in the README's "deviations from spec" section.

ID	QUESTION	POC / UI DOC / BACKEND	RECOMMENDATION	AFFECTS
D1	When a PO rate is edited, do old deliveries change price?	POC reprices all deliveries at the live PO rate. Backend keeps <code>poRateAtSale</code> .	Keep the backend rule. Invoiced deliveries must not change value after the fact. The PO edit form shows: "New rate applies to future deliveries only. 3 existing deliveries keep ₹38,500.00/t."	F1, F7
D2	Must PO numbers be unique across all customers?	POC: globally unique. Backend: unique per customer.	Keep per customer. PO numbers are issued by the buyer, so two buyers can both send "PO-1001". The UI always shows PO number with the customer name.	none
D3	Show an overall average rate difference across all materials?	POC shows overall spread ₹ and %. UI doc forbids averaging margin across copper and HMS.	Show per-material spread as the main view. Keep the overall buy/sell averages, but label the overall difference "Blended (mixed materials)" with an info tooltip and no colour. B12 returns both.	B12, F8
D4	Support units other than tons (kg, pieces)?	POC offers Tons / Kg / Pieces. Backend and UI are tons-only.	Tons only. All maths, fields (<code>quantityTons</code> , <code>ratePerTon</code>) and limits assume tons. Kg can be typed and converted in the input (<code>TonsInput</code> accepts "1250 kg" → 1.250 t) without a data change.	F5
D5	Dashboard: 4 KPI cards (UI doc) or 6 (POC)?	POC adds purchase qty/value, sales qty, total PO qty, extra stock.	Keep 4 cards , and add a compact "More figures" strip under them with Bought, Sold, Total PO, Extra stock and Buy needed. Nothing from the POC is lost and the hierarchy stays.	B8, F7
D6	Where does the frontend keep the refresh token?	Today: JSON body, so the SPA must store it in memory or storage.	httpOnly cookie (B16), switched on by an env flag. Until B16 ships, keep it in memory plus <code>sessionStorage</code> , never <code>localStorage</code> .	B16, F14

Two smaller choices already made in this spec

- **URL filters use ids, not slugs.** The UI doc shows `?supplier=shree-ganesh`, but backend filters take ObjectIds and names can change. URLs use `?companyId=66f...` and the filter chip shows the name from the cached master list. No backend slug field is added (F11).
- **Export is a backend endpoint (B14)**, not client-side paging, so large exports do not hit the 100-row page limit or the rate limiter.

All 31 items

BE backend, **FE** frontend. Source: **G** production guide, **U** UI/UX doc, **P** POC v8, **R** code review.

ID	ITEM	SRC	PRI	SIZE	WHERE
B1	Delivery capacity endpoint <code>GET /sales/capacity</code>	U	P0	M	stock.service, sale.service, sale routes
B2	Totals of the whole filtered set in <code>meta.summary</code>	U	P0	S	purchase, sale, salesPO services + repos
B3	Multi-status filter + <code>GET /sales-pos/summary</code> counts	U P	P0	S	salesPO validation, repo, routes
B4	Whitelisted <code>sort</code> parameter on all lists	U	P0	S	validations/common, all list services
B5	Last purchase rate <code>GET /purchases/last-rate</code>	U	P0	S	purchase service, routes
B6	Wider search (names of supplier, customer, material, PO no.)	U P	P1	S	new utils/search.js, list services
B7	Materials list with stock position (<code>include=position</code>)	U P	P1	S	report.service helper, material service
B8	Dashboard: counts, totals, total PO per material, parties	P U	P1	M	report.service dashboard
B9	Company statistics <code>GET /reports/companies</code> + <code>include=stats</code>	P	P1	M	report.service, company service
B10	Activity feed <code>GET /reports/activity</code>	P U	P1	M	report.service, migration 002
B11	Bought vs delivered trend <code>GET /reports/trend</code>	U	P1	S	report.service
B12	Rate analysis + rate trend <code>GET /reports/rates</code>	P U	P1	M	report.service
B13	Migration 002: <code>createdAt</code> indexes, idempotency collection	R	P1	S	db/migrations
B14	CSV export for purchases, sales POs, sales	U	P2	M	new utils/csv.js, controllers
B15	Demo / seed data script (replaces POC "Reset demo data")	P	P2	S	scripts/seed.js
B16	Refresh token in httpOnly cookie (env-switched)	U	P2	M	auth controller/service, app.js, env
B17	Idempotency-Key on create endpoints	U	P2	M	new middleware + model
F1	Delivery form live limits panel	U	P0	M	features/sales
F2	Lists: summary strip, sortable headers, status tabs with counts	U P	P0	M	DataTable, FilterBar
F3	Purchases list "With supplier now" column	P	P1	S	features/purchases
F4	Sales Orders "Material position" column	P	P1	S	features/sales-orders
F5	Materials page with full position	P U	P1	S	features/materials
F6	Companies list stats + company "Stock used" tab	P	P1	M	features/companies
F7	Dashboard: more figures, company-wise, one material, activity, trend	P U	P1	L	features/dashboard
F8	Rate analysis page (new navigation item)	P	P1	M	features/rates
F9	Complete error dictionary (33 existing API codes + 4 new)	R	P0	S	lib/errors
F10	Query keys and invalidation map	R	P0	S	lib/api-client, hooks
F11	URL filters with ids + name chips	R	P1	S	FilterBar
F12	Export button wired to B14	U	P2	S	PageHeader
F13	Purchase form "last rate" default	U	P0	S	features/purchases

ID	ITEM	SRC	PRI	SIZE	WHERE
F14	Cookie session + Idempotency-Key in the API client	U	P2	S	lib/api-client

Safe daily work (P0)

These five items unblock the UI doc's top priorities: the live delivery limits panel, list summaries, the Open tab and badge, sortable columns, and the "last rate" default. None of them writes data.

B1 Delivery capacity endpoint

P0 BE

GET `/api/v1/sales/capacity?poId&sourceCompanyId&saleDate&excludeSaleId`

Why. The delivery form must show the same limits the server enforces, before the user presses Save (UI doc P0-1, pages 3 and 12-13). Today the numbers only arrive inside a 409 error after a failed save.

Query

PARAM	RULE	MEANING
<code>poId</code>	required, <code>objectId</code>	The sales order being delivered.
<code>sourceCompanyId</code>	optional, <code>objectId</code>	Chosen "Stock from" supplier. Without it the supplier limit is omitted and only the <code>sources</code> list is returned.
<code>saleDate</code>	optional, <code>businessDate</code>	Defaults to <code>businessToday(BUSINESS_TIMEZONE)</code> .
<code>excludeSaleId</code>	optional, <code>objectId</code>	When editing a delivery, that delivery is not counted against its own limits (same rule as <code>assertCapacity</code>).

The key rule: use the ledger floor, not only the balance on the day

A save is accepted only if (1) quantity $\leq$ PO remaining, $\leq$ yard stock *as of the sale date*, $\leq$ supplier stock *as of the sale date*, **and** (2) the ledger replay after the save never goes negative on any later day (`assertMaterialLedgerNonNegative` / `assertSourceLedgerNonNegative`). For a backdated delivery, (2) can be stricter than (1). So the endpoint returns the **floor**: the lowest balance on any day from `saleDate` onwards. The floor is never higher than the balance on the day, so `maxAllowedTons` = min(PO remaining, yard floor, supplier floor), floored at zero. That is exactly the largest quantity the server will accept.

Response (200)

```
{ "success": true, "data": {
  "saleDate": "2026-09-27",
  "po": { "_id": "66f...142", "poNumber": "PO-0142", "poDate": "2026-09-09",
    "company": { "_id": "66f...b01", "name": "Bharat Castings Pvt Ltd" },
    "material": { "_id": "66f...201", "name": "HMS 1" },
    "ratePerTon": "38500.00", "quantityTons": "30.000", "deliveredTons": "18.250",
    "lifecycleStatus": "ACTIVE", "displayStatus": "PARTIALLY_SUPPLIED" },
  "limits": {
    "poRemainingTons": "11.750",
    "yardStockTons": "71.500",           // balance at end of saleDate
    "yardFloorTons": "71.500",         // lowest balance on saleDate or any later day
    "sourceStockTons": "9.200",        // null when sourceCompanyId is absent
    "sourceFloorTons": "9.200"
  },
  "maxAllowedTons": "9.200",
  "bindingLimit": "SOURCE_STOCK",      // PO_REMAINING | YARD_STOCK | SOURCE_STOCK
  "blockers": [],                     // see table below; non-empty means maxAllowedTons = "0.000"
  "sources": [ { "sourceCompanyId": "66f...c11", "name": "Shree Ganesh Metals",
    "availableTons": "9.200", "floorTons": "9.200" }, ... ],
  "hiddenEmptySources": 2
} }
```

Blockers (returned, not thrown)

CODE	WHEN
PO_NOT_AVAILABLE	PO lifecycle is CANCELLED (unless editing a sale already on this PO without raising quantity; the form handles that).
PO_COMPLETED new, read-only	PO remaining is 0.
SALE_BEFORE_PO_DATE	<code>saleDate < po.poDate</code> .
INVALID_SOURCE_COMPANY	Supplier is inactive or not PURCHASE/BOTH.
INVALID_SALE_COMPANY / INVALID_MATERIAL	Customer or material on the PO has been deactivated.

Blockers are returned in the 200 body so the panel can explain them without an error state. Only a missing PO (404 `PO_NOT_FOUND`) and bad input (422) are thrown.

Implementation

- stock.service.js** : add `ledgerFloor(openingBalance, inflowDays, outflowDays, fromDate)` next to `findFirstNegativeDay`. It replays the same day map and returns the minimum balance over all days $\geq$ `fromDate`, starting with the balance at end of `fromDate`. Add `getMaterialFloor(materialId, fromDate, {excludeSaleId})` and `getSourceFloors(materialId, fromDate, {excludeSaleId, sourceCompanyId?})`. The second uses **two** aggregations grouped by `(company, day)` for purchases and `(sourceCompanyId, day)` for sales of that material, then replays each supplier in memory. That is one round trip for every supplier.
- sale.service.js** : extract the body of `assertCapacity` into `computeCapacity({ po, sourceCompanyId, saleDate, excludeSaleId, session })`, returning the limits above. `assertCapacity` calls it and throws exactly the same three codes as today. The 409 `details` gain `yardFloorTons / sourceFloorTons`, and `maxAllowedTons` becomes floor-based. That is additive and more accurate. Add `saleService.capacity(query)` : load the PO with `salesPORepository.findWithPosition`, collect blockers, call `computeCapacity` without a session (read-only), attach `sources`.
- sale.validation.js** : `saleCapacitySchema`. **sale.controller.js** : `saleCapacity`. **sale.routes.js** : `router.get('/capacity', validate(saleCapacitySchema), asyncHandler(saleCapacity))` placed **above** `('/:id')`.

Race with other users

Capacity is a read. Another save can land between the read and the user's Save. The server still enforces every limit inside the transaction, so this cannot corrupt data. The frontend refetches capacity on any 409 and after a successful save (F1).

Tests (tests/integration/sales-capacity.test.js)

- Parity: create a sale at exactly `maxAllowedTons` $\rightarrow$ 201; at `maxAllowedTons + 0.001` $\rightarrow$ 409. Run it three times, once for each binding limit.
- Backdated case: a later delivery exists, so `yardFloorTons < yardStockTons`. Saving above the floor returns `NEGATIVE_STOCK_HISTORY`; saving at the floor succeeds.
- `excludeSaleId` : editing a 5 t sale on a PO with 0 t left reports 5 t available.
- Blockers: cancelled PO, completed PO, date before PO, inactive supplier. Each gives 200, a non-empty `blockers` list and `"0.000"`.
- `/sales/capacity` is not swallowed by `('/:id')`. VIEWER gets 200. Unknown query key $\rightarrow$ 422.

B2 Totals for the whole filtered list

PO BE

Why. The Purchases mockup shows "6 purchases · 54.195 t · ₹22,15,130.00" computed on the server, because a page-only total would mislead (UI doc page 9). Deliveries and Sales Orders need the same.

Contract (additive to existing list responses)

```
// GET /api/v1/purchases?from=2026-09-01&to=2026-09-30&companyId=66f...c11
"meta": { "page": 1, "limit": 25, "total": 6, "totalPages": 1,
  "summary": { "quantityTons": "54.195", "totalAmount": "2215130.00", "averageRate": "40873.33" } }

// GET /api/v1/sales ... same shape: quantityTons, totalAmount, averageRate
// GET /api/v1/sales-pos ... quantityTons, deliveredTons, remainingTons, totalPOAmount
```

`averageRate` is the weighted average ($\text{totalAmount} \div \text{quantityTons}$, 2 dp, `"0.00"` when there is no quantity). `total` already carries the count, so it is not repeated.

Implementation

- `base.repository.js`: add `summarize(match, fields)`, which is `groupTotals(match, null, fields)` returning one row of Decimals (zeros when empty).
- `purchase.service.js` / `sale.service.js` `list()`: build the filter with `toObjectId()` for every id (the current filter passes strings, which `find()` casts but `aggregate()` would not). Run `paginate` and `summarize` in `Promise.all`. Return `{ items, page, limit, total, summary }`.
- `salesPO.repository.js` `listWithPosition`: add a `summary` branch to the existing `$facet`. It must run **after** `positionStages`, because delivered and remaining are computed, so the unfiltered branch applies `positionStages` inside the summary facet only.
- Controllers: `ok(res, result.items, { ...pageMeta(result), summary: result.summary })`. `pageMeta` is unchanged.

Tests: summary equals the sum of all pages (seed 30 rows, `limit=10`); a filter by company changes the summary; an empty result gives zero strings; decimals stay exact ($0.1 + 0.2 = "0.300"$).

B3 Multi-status filter and Sales Order status summary

PO BE

GET `/api/v1/sales-pos?status=PENDING,PARTIALLY_SUPPLIED`

GET `/api/v1/sales-pos/summary?companyId&materialId&from&to`

Why. The Open tab, the dashboard drill-down and the sidebar badge all mean "Pending + Partly delivered" (UI doc pages 5 and 8). The delivery form must list only open POs for the chosen customer (POC `refreshSalePOs`). Tabs show counts ("Open · 14, Partly delivered · 5").

Filter change

`status` accepts one value or a comma-separated list. It is backward compatible, because a single value still works. The alias `status=OPEN` expands to `PENDING,PARTIALLY_SUPPLIED`.

```
// validations/salesPO.validation.js
const displayStatusList = z.string().transform((v) => v.split(',').map((s) => s.trim().toUpperCase()))
  .transform((list) => list.flatMap((s) => (s === 'OPEN' ? [PENDING, PARTIALLY_SUPPLIED] : [s])))
  .pipe(z.array(z.enum(Object.values(PO_DISPLAY_STATUS))).min(1).max(4));
// repository: { $match: { displayStatus: { $in: statuses } } } replaces { $match: { displayStatus } }
```

Summary endpoint response

```
{ "counts": { "PENDING": 9, "PARTIALLY_SUPPLIED": 5, "COMPLETED": 31, "CANCELLED": 2, "OPEN": 14, "ALL": 47 },
  "open": { "remainingTons": "212.300", "oldestPoDate": "2026-09-02" } }
```

One aggregation: `$match` (same filter builder as `list`, ids cast), `positionStages`, then `$group` by `displayStatus`. Route `/summary` goes above `/:id`. **Tests:** list with two statuses; `OPEN` alias; invalid status → 422; counts add up to `ALL`; summary respects `companyId`.

B4 Server-side sorting

PO BE

GET /api/v1/purchases?sort=-totalAmount

Why. The DataTable runs in server mode with sortable headers and `aria-sort` (UI doc page 10). "Orders waiting, oldest first" on the dashboard needs `sort=poDate`. Today every list has a fixed order.

`sort` takes one field, optionally prefixed with `-` for descending. The default is today's order, so nothing changes when `sort` is absent. `_id` is always appended as a tie-breaker so paging is stable. Decimal128 fields sort numerically in MongoDB.

LIST	ALLOWED FIELDS	DEFAULT
purchases	purchaseDate quantityTons ratePerTon totalAmount createdAt	-purchaseDate
sales	saleDate quantityTons totalAmount createdAt	-saleDate
sales-pos	poDate poNumber quantityTons ratePerTon remainingQuantityTons	-poDate
companies	name createdAt	name
materials	name createdAt	name

```
// validations/common.js
export const sortQuery = (fields) => z.string()
  .regex(new RegExp(`^-${fields.join('|')} $`), 'sort must be one of: ${fields.join(', ')} (prefix - for descending)')
  .transform((v) => (v.startsWith('-') ? { [v.slice(1)]: -1 } : { [v]: 1 }));
// service: sort: { ...(query.sort ?? DEFAULT_SORT), _id: -1 }
```

Sales POs: sorting by `remainingQuantityTons` needs the position first. In `listWithPosition`, when that sort is requested, take the branch that runs `positionStages` before `$sort` (the same branch the status filter uses). **Tests:** each list asc/desc, invalid field → 422, tie-break stable across pages.

B5 Last purchase rate

PO BE

GET /api/v1/purchases/last-rate?companyId&materialId

Why. Smart defaults ("last supplier / material / rate") make a purchase a 2-click task (UI doc P0-3, page 30 "High").

```
{ "ratePerTon": "34500.00", "purchaseDate": "2026-09-27", "purchaseId": "66f...", "scope": "SUPPLIER_MATERIAL" }
// scope: SUPPLIER_MATERIAL (both given, match found) | MATERIAL (fell back to any supplier) | null result
// data is null when the material has never been bought
```

`materialId` required, `companyId` optional. Query:

`Purchase.findOne({companyId, materialId}).sort({purchaseDate:-1, createdAt:-1}).select('ratePerTon purchaseDate')` served by the existing `{companyId, materialId, purchaseDate}` index. If nothing matches, fall back to material only. Route above `/:id`. **Tests:** exact match, fallback, never bought → `data: null`, backdated purchase does not win over a newer business date.

POC parity (P1)

These items bring back what the POC showed and the production plan dropped: material position everywhere, company-wise figures, rate analysis, activity and trends. All of them are reads built on the existing report service, so every number follows the activity-vs-position rule.

B6 Wider search

P1 BE

Why. The POC searched purchases by material, supplier, invoice and vehicle, and sales by customer, "stock from" supplier, PO number, material, challan and vehicle. The UI doc's search box says "Supplier, vehicle or invoice no." The backend only searches invoice/challan and vehicle.

LIST	SEARCH MATCHES (CASE-INSENSITIVE, CONTAINS)
purchases	invoice no., vehicle no., supplier name, material name
sales	challan no., vehicle no., customer name, stock-from supplier name, material name, PO number
sales-pos	PO number, customer name, material name
companies	name, GST, contact phone, contact person, contact email, address (city)
materials	name, notes

Implementation

- New `src/utils/search.js`: `idsMatching(repository, text, field = 'name', cap = 200)` returns ObjectIds of masters whose name matches (masters are small; capped). `codeRegex(text) = escapeRegex(normalizeCode(text))`.
- Each list service builds `$or` from these: e.g. purchases
`[{normalizedInvoiceNumber: codeRegex}, {vehicleNumber: codeRegex}, {companyId: {$in: supplierIds}}, {materialId: {$in: poIds}}` from a PO-number lookup.
- Bug fix included:** vehicle and invoice numbers are stored in code form (`GJ03AX4521`), but search uses `normalizeText`, so typing "GJ 03 AX" finds nothing today. Code fields must be searched with `codeRegex`.
- Search requires at least 2 characters (Zod `.min(2)`); shorter input is ignored by the frontend.

Scale note: contains-regex scans are fine at yard scale (tens of thousands of rows). If lists grow past ~200k rows, add a MongoDB text index; the API contract does not change. **Tests:** one per new field; "GJ 03 AX" matches `GJ03AX4521`; regex characters in input are escaped; search combines with the other filters (AND).

B7 Materials with stock position

P1 BE

GET `/api/v1/materials?include=position&asOf=2026-09-27`

Why. UI doc Materials columns are "In yard · Open demand · Buy needed". The POC material cards show purchased, total PO, sold, remaining PO, current stock and final position. The POC Sales PO list shows each row's material position. Today `GET /materials` returns master data only.

```
// each item gains (only when include=position):
"position": { "asOf": "2026-09-27", "currentStockTons": "71.500", "openDemandTons": "96.000",
              "purchaseRequiredTons": "24.500", "extraStockTons": "0.000",
              "activePOQuantityTons": "150.000", "status": "SHORT" } // SHORT | EXTRA | COVERED
```

- Extract `materialPositions({ asOf, materials })` from `reportService.dashboard`. It wraps `stockByMaterialAsOf` and `poPositionsAsOf` restricted to the given materials. The dashboard then calls the helper too, so both endpoints can never disagree.

- `material.service.list`: after `paginate`, when `include=position`, call the helper for the page's materials only. `GET /materials/:id` keeps `currentStockTons` and also gains `position`.
- Validation: `include: z.enum(['position']).optional()`, `asOf: businessDate.optional()`.

Tests: position equals the dashboard row for the same material and date; cancelled POs are not counted as demand; `asOf` in the past ignores later purchases.

B8 Dashboard extensions

P1 BE

GET `/api/v1/reports/dashboard?from&to&materialId&companyId&include=parties`

Why. The POC dashboard shows purchase/sales quantities and values, total PO quantity, extra stock and, per material, who it was bought from and sold to. The UI doc mockup shows "380.860 t in 41 deliveries", "9 pending · 5 partly delivered" and "3 materials are short". All are additive fields.

NEW FIELD	MEANING
<code>purchases.count</code> , <code>sales.count</code>	Records in the period (activity).
<code>po.pendingPOCount</code> , <code>po.partlyDeliveredPOCount</code>	Split of <code>openPOCount</code> by delivered = 0 or > 0 (as of <code>to</code>).
<code>totals.stockInYardTons</code>	Sum of <code>currentStockTons</code> (position).
<code>totals.purchaseRequiredTons</code> , <code>totals.extraStockTons</code> , <code>totals.shortMaterialCount</code>	Sums and count over material rows (POC "Purchase required" and "Extra stock" cards).
<code>totals.netStockChangeTons</code>	Purchased – sold in the period ("▲ 31.520 t since 01/09").
<code>materials[].activePOQuantityTons</code> , ... <code>activePOValue</code> , ... <code>deliveredOnActivePOTons</code>	POC "Total PO qty" and "PO value" per material (active POs as of <code>to</code>).
<code>materials[].suppliers[]</code> , ... <code>customers[]</code> (include=parties)	<code>{companyId, name, quantityTons, value}</code> from period purchases / sales grouped by (material, company). Opt-in to keep the default payload small.

Comparison ("▲ 8% vs Aug") stays a second dashboard call from the frontend with the previous range (F7). No backend change is needed, and both periods keep identical semantics. **Tests:** totals equal sums of rows; counts match inserted records; parties grouped correctly; existing dashboard tests unchanged.

B9 Company statistics

P1 BE

GET `/api/v1/reports/companies?from&to&type&search&sort&limit`

GET `/api/v1/companies?include=stats&from&to` (same numbers merged into the paged list)

Why. POC dashboard "Company-wise purchases / sales" panels and the POC companies table columns: purchases, purchase stock used, purchase stock available, POs, sales. Neither exists today; `GET /reports/companies/:id` covers one company only.

```
{ "companyId": "66f...c11", "name": "Shree Ganesh Metals", "type": "PURCHASE", "isActive": true,
  "purchases": { "count": 6, "quantityTons": "54.195", "value": "2215130.00" }, // activity, as supplier
  "sales": { "count": 0, "quantityTons": "0.000", "value": "0.00" }, // activity, as customer
  "orders": { "activeCount": 0, "openCount": 0, "remainingTons": "0.000" }, // position, as customer
  "sourcePool": { "purchasedTons": "212.400", "usedForSalesTons": "188.300",
    "availableTons": "24.100", "usedInPeriodTons": "41.550" } } // position as of to
+ period use
```

- `reportService.companyStats({ companyId?, from, to })` runs five grouped aggregations (purchases by `companyId`, sales by `companyId`, sales by `sourceCompanyId` in period, pool totals as of `to`, PO positions by

customer) and merges them in memory. Both endpoints call it.

- `type` filters to PURCHASE, SALE or BOTH (BOTH companies appear under either side). `sort`: `name`, `-purchaseValue`, `-salesValue`, `-availableTons`. `limit` ≤ 500 (dashboard panels use `limit=8`).
- The pool is **all-time as of** `to`, like `/reports/source-stock`. The POC mixed an all-time pool with period purchases in one row; here the two are separate, labelled fields.

Tests: BOTH company appears with purchase and sales figures; pool equals `/reports/source-stock` summed per company; `include=stats` only aggregates the returned page.

B10 Activity feed

P1 BE

GET `/api/v1/reports/activity?limit=20&cursor&types=PURCHASE,SALE,SALES_PO`

Why. Both the POC and the UI doc show "Recent activity" (purchases, deliveries and new orders, newest first). Lists are ordered by business date, so a backdated entry would never show as recent; the feed orders by `createdAt`.

```
{ "items": [ { "type": "SALE", "id": "66f...", "at": "2026-09-27T10:42:11.201Z", "date": "2026-09-26",
  "company": { "_id": "...", "name": "Rajkot Alloys" }, "material": { "_id": "...", "name": "MS turnings" },
  "quantityTons": "15.000", "amount": "532500.00", "reference": "CH-1044",
  "po": { "_id": "...", "poNumber": "PO-0138", "displayStatus": "COMPLETED" },
  "sourceCompany": { "_id": "...", "name": "Patel Scrap Co." } }, ... ],
  "nextCursor": "eyJhdCI6IjYyJy..."} // null when there is nothing older
```

- Three queries, `find({ createdAt/_id before cursor }).sort({ createdAt: -1, _id: -1 }).limit(limit)` with the display populates, merged and cut to `limit`. The cursor is base64 JSON `{at, id}`, so equal timestamps page correctly.
- `po.displayStatus` is the order's status *now* ("now Completed" in the mockup), via `salesPORepository.findWithPosition` for the POs on the page only.
- `limit` 1–50, default 20. Needs the `createdAt` indexes from B13.

Scope: the feed shows **creations**. Edits are not listed because the system has no audit log. An `audit_events` collection written by each service is a later item; this feed's contract can absorb a new `type` such as `PURCHASE_EDITED` without changes.

B11 Bought vs delivered trend

P1 BE

GET `/api/v1/reports/trend?from&to&bucket=week&materialId&companyId`

Why. The UI dashboard chart "Bought vs delivered per week (t)" answers "are we buying faster than we deliver?".

```
{ "bucket": "week", "from": "2026-09-01", "to": "2026-09-27",
  "series": [ { "start": "2026-09-01", "end": "2026-09-06", "purchasedTons": "96.420", "purchaseValue": "...",
    "soldTons": "88.100", "salesValue": "..."},
    { "start": "2026-09-07", "end": "2026-09-13", ... }, ... ] }
```

- `bucket`: `day` | `week` (Monday start, as in the mockup: 07–13 Sep) | `month`. Defaults when `from` is absent: last 8 weeks, last 30 days or last 12 months, ending at `to` or business today.
- Two aggregations using `$dateTrunc: { date, unit, startOfWeek: 'monday', timezone: 'UTC' }`. UTC is correct because business dates are stored at UTC midnight. Missing buckets are zero-filled in code; the first and last buckets are clipped to `from / to`. Requires MongoDB ≥ 5.0 (Atlas and the test replica set both qualify).
- Maximum 400 buckets, otherwise 422 `VALIDATION_ERROR` ("choose a larger bucket").

GET `/api/v1/reports/rates?from&to&materialId&bucket`

Why. The POC "Rate Analysis" page (overall and material-wise weighted average buying and selling price, difference in ₹ and %, a "check one material" view), plus the UI doc's rate trend chart (page 30, "Medium").

```
{ "period": { "from": "2026-09-01", "to": "2026-09-30" },
  "overall": { "purchasedTons": "...", "purchaseValue": "...", "averageBuyingRate": "36120.40",
    "soldTons": "...", "salesValue": "...", "averageSellingRate": "41210.75",
    "difference": "5090.35", "differencePct": "14.09", "blended": true },
  "materials": [ { "materialId": "...", "materialName": "HMS 1", "purchasedTons": "...", "purchaseValue": "...",
    "averageBuyingRate": "34310.00", "soldTons": "...", "salesValue": "...",
    "averageSellingRate": "38500.00", "difference": "4190.00", "differencePct": "12.21" }, ... ],
  "series": [ { "start": "...", "end": "...", "averageBuyingRate": "...", "averageSellingRate": "..." }, ... ] } //
only with bucket + materialId
```

- All averages are weighted (value ÷ tons) with `avgRate()`. `difference` and `differencePct` are `null` unless both bought and sold tons are > 0 in the period (POC `hasBoth`). `differencePct` = difference ÷ averageBuyingRate × 100, 2 dp.
- Selling value uses the stored `totalAmount`, which is the rate actually billed (`poRateAtSale`), not the live PO rate. This follows decision D1 and differs from the POC.
- `overall.blended: true` tells the UI to label the overall difference "Blended (mixed materials)" (decision D3).
- `series` reuses B11's bucketing; it requires `materialId`, because a series across materials would be the blended number again.

Tests: weighted average vs simple average differ and the weighted one is returned; one-sided material gives null difference; rounding at 2 dp; the series bucket average equals the period average when there is one bucket.

B13 Migration 002

New file `src/db/migrations/002-activity-and-idempotency.js`, appended to the list in `migrations/index.js` (never edit 001).

```
export const id = '002-activity-and-idempotency';
export const description = 'createdAt indexes for the activity feed; idempotency key collection';
export async function up() {
  for (const model of [Purchase, SalesPO, Sale, IdempotencyKey]) {
    await model.createCollection();
    await model.syncIndexes();
  }
}
```

Declare every index in the schema, not only in the migration

Migration 001 uses `syncIndexes()`, which **drops** any index not declared in the Mongoose schema. So

`schema.index({ createdAt: -1, _id: -1 })` is added to `Purchase`, `SalesPO` and `Sale`, and 002 syncs them. An index created only inside the migration would be silently removed by the next sync.

The `IdempotencyKey` model belongs to B17. If B17 is postponed, ship 002 without it and add 003 later. **Tests:** `database.test.js` asserts the new indexes exist and that running the migrator twice is a no-op.

Convenience and hardening (P2)

Export and demo data make the app pleasant to run; the cookie session and idempotency keys make it safer on shared devices and weak yard connections. Each is switched on without breaking current clients.

B14 CSV export

P2 BE

GET `/api/v1/purchases/export.csv` · `/sales-pos/export.csv` · `/sales/export.csv`

Why. The list header's "Export" exports the current filters (UI doc page 9). Client-side paging would hit the 100-row page cap and the per-minute rate limit.

- Takes exactly the list query (filters, `search`, `sort`) without `page` / `limit`, and reuses the list service's filter builder, so the export always matches the screen.
- Counts first: more than 50,000 rows → 422 `EXPORT_TOO_LARGE` with `details.total` ("Narrow the date range"). Otherwise it streams with a Mongoose cursor. Headers: `Content-Type: text/csv; charset=utf-8`, `Content-Disposition: attachment; filename="purchases_2026-09-01_2026-09-30.csv"`. It starts with a UTF-8 BOM so Excel shows ₹ and Hindi names correctly.
- Values: dates `YYYY-MM-DD`, decimals as exact plain strings (no thousands separators, so Excel sums work), names instead of ids, and ids in a trailing column for re-import.
- **CSV injection guard:** any cell starting with `= + - @` or a tab is prefixed with `'`. New `src/utls/csv.js` holds `csvRow()` and escaping (quotes doubled, fields with comma, quote or newline wrapped).
- This is the one endpoint outside the JSON envelope; errors before streaming still use the envelope. Open to VIEWER.

FILE	COLUMNS
purchases	Date, Supplier, Material, Tons, Rate per ton, Amount, Vehicle, Invoice, Notes, Purchase ID
sales-pos	PO no., PO date, Customer, Material, Ordered t, Delivered t, Left t, Rate per ton, PO amount, Status, PO ID
sales	Date, PO no., Customer, Stock from, Material, Tons, Rate per ton (at sale), Amount, Vehicle, Challan, Sale ID

Tests: header and row count match the list total; a filter is applied; the injection guard works; 50,001 rows gives 422 (use a lowered limit via an env override in the test).

B15 Demo / seed data script

P2 BE

CLI `npm run seed -- --demo [--reset]`

Why. Replaces the POC's "Reset demo data" button. It is useful for staging, demos, frontend development and user testing (UI doc page 30). It is never a production feature, and there is no HTTP endpoint for it.

- `scripts/seed.js`, `"seed": "node scripts/seed.js"` in `package.json`. Refuses to run when `NODE_ENV=production`. Refuses when purchases, POs or sales already exist, unless `--reset` is given.
- `--reset` additionally requires `ALLOW_DB_RESET=true`. It clears companies, materials, purchases, POs, sales and idempotency keys, **never** users or refresh sessions.
- Writes **through the services** (`companyService.create`, `purchaseService.create`, ...), so every validation, ledger check and amount calculation applies exactly as in the app. `createdBy` is the first active OWNER; if there is none, the script tells the user to run `npm run create-owner` first.
- Data: the POC demo set (4 materials, 4 companies incl. one BOTH, 3 purchases, 3 sales POs, 2 deliveries with "stock from" suppliers), with dates relative to business today (-7 ... -2 days). The Brass PO has no purchase, so the dashboard shows a real "Buy needed" case.

Why. A refresh token in JSON has to be stored by page scripts, so any XSS can steal a 30-day session. An httpOnly cookie is never readable by JavaScript (UI doc page 30, decision D6).

Switch

ENV VAR	DEFAULT	MEANING
<code>AUTH_REFRESH_TRANSPORT</code>	<code>body</code>	<code>body</code> = today's behaviour · <code>cookie</code> = cookie only · <code>both</code> = transition period (accept either, send both)
<code>REFRESH_COOKIE_NAME</code>	<code>msm_rt</code>	Cookie name.
<code>COOKIE_SECURE</code>	<code>true</code>	Must be <code>true</code> in production (env validation enforces it).
<code>COOKIE_SAMESITE</code>	<code>strict</code>	<code>strict</code> when app and API are on the same site (<code>app.metalix.in</code> + <code>api.metalix.in</code>). <code>none</code> only if they are on different sites.
<code>COOKIE_DOMAIN</code>	<code>unset</code>	Optional parent domain.

Behaviour in cookie mode

- **Login / refresh** set `Set-Cookie: msm_rt=...; HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth; Max-Age=REFRESH_TOKEN_TTL_DAYS` and omit `refreshToken` from the JSON body. The rotation logic in `authService.refresh` is unchanged.
- **Refresh / logout** read the token from the cookie. `refreshSchema` body becomes optional in cookie mode. Logout clears the cookie (`Max-Age=0`).
- **CSRF:** when the token comes from a cookie, `/auth/refresh` and `/auth/logout` require the header `X-Requested-With: metalix` and an `Origin` in `CORS_ORIGINS`. Otherwise → 403 `CSRF_CHECK_FAILED`. Cross-site forms cannot set custom headers, and CORS blocks cross-origin fetches.
- **CORS:** `credentials: true` and `allowedHeaders` include `X-Requested-With` and `Idempotency-Key`. Env validation rejects `CORS_ORIGINS=*` in cookie mode.
- Cookie parsing: add `cookie-parser` (one small, widely used dependency), mounted only on the auth router.

Tests: login sets the cookie and omits the body token; refresh via cookie rotates the cookie and the old one fails; missing header → 403; `both` mode accepts body or cookie; `body` mode matches today's tests byte for byte.

B17 Idempotency keys on create endpoints

POST `/companies` · `/materials` · `/purchases` · `/sales-pos` · `/sales` + header `Idempotency-Key`

Why. On a flaky yard connection the request may succeed while the response is lost. "Try again" then creates a second purchase. With a key, the retry returns the first result instead (UI doc page 30).

Rules

- The header is optional; without it, behaviour is unchanged. Format: 8–100 chars `[A-Za-z0-9_-]` (the frontend sends a UUID v4).
- New model `IdempotencyKey`:

```
{
  userId,
  key,
  method,
  path,
  requestHash,
  state: IN_PROGRESS|COMPLETED,
  responseStatus,
  responseBody,
  expiresAt
}
```

 unique `(userId, key)`, TTL on `expiresAt` (24 h). Created in migration 002/003.
- Middleware `src/middleware/idempotency.js`, placed after `authorize` and before `validate` on the five routes:

SITUATION	RESULT
New key	Insert <code>IN_PROGRESS</code> , run the handler. On 2xx store <code>COMPLETED</code> + response. On any error delete the record, so a corrected retry can run.

SITUATION**RESULT**

Same key, same path + body hash, COMPLETED	Replay the stored status and body with header <code>Idempotent-Replayed: true</code> . Nothing is written.
Same key, still IN_PROGRESS	409 <code>IDEMPOTENCY_IN_PROGRESS</code> with <code>Retry-After: 2</code> .
Same key, different body or path	422 <code>IDEMPOTENCY_KEY_REUSED</code> .
IN_PROGRESS older than 60 s	Treated as abandoned (process crashed) and taken over.

Known limit: if the process dies after the business transaction commits but before the response is stored, a retry more than 60 s later can create a duplicate. Closing that fully means writing the key inside each service transaction, which is a larger change. The middleware already covers the common case: a lost response on a live server.

Tests: two identical POSTs with one key create one purchase and the second has the replay header; a different body → 422; concurrent same key → one 201 and one 409; a failed request (409 stock) can be retried with the same key after stock arrives.

Screen changes F1–F8

These changes are added to the UI/UX System v1.0 (Vite · React 19 · shadcn/ui on Base UI · TanStack Router/Query/Table · RHF + Zod). All of its rules still apply: numbers through `lib/format.ts`, write actions hidden for VIEWER, filters in the URL, old rows kept while loading.

Navigation after this spec

One new module, **Rate analysis**, joins "Stock & reports". There are now nine modules; everything else stays in place.

OVERVIEW	DAILY WORK	STOCK & REPORTS	SETUP
Dashboard	Purchases · Sales Orders (badge from B3) · Deliveries	Supplier stock · Company report · Rate analysis (new, F8)	Companies · Materials

F1 Delivery form: live limits panel

P0FE

uses B1

- `useCapacity({ poId, sourceCompanyId, saleDate, excludeSaleId })`: TanStack Query, key `['capacity', ...]`, refetched 300 ms after PO, supplier or date change (debounced), with `placeholderData: keepPreviousData`.
- "Stock from" options come from `capacity.sources` (sorted by `floorTons`, with a "2 suppliers with no stock are hidden" note). This replaces a separate source-stock call.
- Three meters (typed ÷ each limit). The smallest limit is ringed and tagged "Tightest". One sentence names `bindingLimit` in trade words.
- When `yardFloorTons` < `yardStockTons`, add: "X t is already promised to later deliveries."
- Quantity defaults to `maxAllowedTons`; "Use max" refills it. Blockers replace the panel with one plain sentence and the fix ("Change order", "Choose another supplier").
- On any 409 from save: refetch capacity, keep the form, apply the F9 message.

Delivery limits

PO-0142 · HMS 1 · 27/09/2026

Left on order11.750 t

In yard71.500 t

Shree Ganesh Metals stockTightest9.200 t

You can deliver up to 9.200 t

Limited by Shree Ganesh Metals stock

F2 Lists: summary strip, sortable headers, status tabs with counts

P0FE

uses B2 B3 B4 B6

- Summary strip** above every transaction table, from `meta.summary`: "6 purchases · 54.195 t · ₹22,15,130.00 · avg ₹40,873.33/t". It shows the whole filtered set, never the page.
- Sortable headers** map to `?sort=` (URL-held). Only B4's whitelisted columns get the sort affordance.
- Sales Orders tabs** "Open · 14 | Partly delivered · 5 | Completed | All", counts from `/sales-pos/summary`. Open = `status=OPEN`. The sidebar badge uses the same query (`counts.OPEN`).
- Search placeholder** lists what B6 searches: "Supplier, material, vehicle or invoice no."

F3 Purchases: "With supplier now"

P1

Optional column (behind "Columns", off by default on phones): the current pool for that row's supplier + material, e.g. **24.100 t** with the muted line "HMS 1 left from Shree Ganesh". Filled from one `/reports/source-stock` call per page, mapped by `sourceCompanyId:materialId`. It is position *today*, so the header tooltip says so. This is the POC "Current supplier stock" column.

F4 Sales Orders: "Material position"

P1

Column after "Left (t)": the order's material position from `/materials?include=position` (one call, cached 60 s). A **Short 24.500 t** badge links to a new purchase with that material filled in; **Extra 6.900 t**; "Covered". This is the POC "Purchase balance" column, also shown in the dashboard's "Orders waiting".

F5 Materials page with full position

P1 FE uses B7 B8

The table keeps the UI doc columns (**Name · In yard · Open demand · Buy needed**) from `include=position`. Clicking a row opens a sheet with the POC card: Purchased, Total PO, Sold, Remaining PO, Current stock, Purchase value and the final position sentence, from `/reports/dashboard?materialId=...` for the global period. Units are tons only (D4). The "Unit" field from the POC is not shown.

F6 Companies list stats and "Stock used" tab

P1 FE uses B9

- The list calls `/companies?include=stats&from&to`. Default columns: Name · Type · Phone · City · **Bought (t) · Sold (t) · Stock with us (t)**. GSTIN, open orders and "Used for sales" sit behind "Columns". Supplier-only columns show "—" for SALE companies, and vice versa.
- Company detail tabs become **Purchases · Orders · Deliveries · Stock used · Stock**. "Stock used" = `/sales?sourceCompanyId=...` (deliveries that consumed this supplier's stock: date, customer, PO, material, tons), the POC's "Stock used from this purchase company" table.

F7 Dashboard additions

P1 FE uses B7–B11

Dashboard · This month

01/09/2026 – 27/09/2026 · compared with 01/08/2026 – 27/08/2026

STOCK IN YARD

286.5 t

▲ 31.520 t since 01/09

BUY NEEDED

38.8 t

3 materials are short

OPEN SALES ORDERS

14

9 pending · 5 partly delivered

DELIVERED THIS MONTH

₹1.57 Cr

▲ 8% vs Aug · 380.860 t

More figures: Bought **412.380 t** · ₹1.49 Cr Sold **380.860 t** Total PO **596.000 t** Extra stock **112.900 t** Buy needed **38.750 t**

BLOCK	DATA	POC / UI ORIGIN
"More figures" strip (D5)	<code>purchases</code> , <code>sales</code> , <code>po.poQuantityTons</code> , <code>totals.*</code>	POC 6 KPI cards
Comparison "▲ 8% vs Aug"	Second dashboard call with the previous range: same day-count, month-aligned for "This month" (01–27 Aug)	UI p.7
Needs attention	Buy needed: <code>materials[]</code> with <code>purchaseRequiredTons</code> > 0 · Orders waiting: <code>/sales-pos?status=OPEN&sort=poDate&limit=3</code> · Low supplier stock: <code>/reports/source-stock</code> rows under 1 t where the material has open demand	UI p.7
Stock vs open orders bars	<code>materials[]</code> <code>currentStockTons</code> vs <code>remainingPOQuantityTons</code> , sorted by shortage	UI p.7
Material-wise position (table view)	<code>materials[]</code> with <code>include=parties</code> : bought from / PO customers / sold, current stock, remaining, final balance, status	POC main table
View one material	Material picker → <code>/reports/dashboard?materialId</code> , six position cards	POC
Company-wise purchases / sales	<code>/reports/companies?type=PURCHASE&sort=-purchaseValue&limit=8</code> and the SALE equivalent	POC
Bought vs delivered per week	<code>/reports/trend?bucket=week</code> , two labelled lines	UI p.7
Recent activity	<code>/reports/activity?limit=8</code> , "Show more" pages with <code>nextCursor</code>	POC + UI

Above the fold, the UI doc rule still holds: 4 cards and at most 2 charts. The material table, company-wise panels and "view one material" sit below the fold, in that order.

F8 · Rate analysis

The POC page, rebuilt on B12 with decision D3: material rows are the real answer, and the overall difference is labelled as blended.

Rate analysis

Weighted averages: total value ÷ total tons. Selling rates are the rates actually billed on each delivery. · This month ▾

AVG BUYING RATE

₹36,120.40

412.380 t bought · ₹1.49 Cr

AVG SELLING RATE

₹41,210.75

380.860 t sold · ₹1.57 Cr

BLENDED DIFFERENCE ①

₹5,090.35

mixed materials, see table

BLENDED DIFFERENCE %

14.09%

of avg buying rate

MATERIAL	BOUGHT (T)	AVG BUY (₹/T)	SOLD (T)	AVG SELL (₹/T)	DIFFERENCE (₹/T)	%
HMS 1	96.450	34,310.00	88.200	38,500.00	+4,190.00	+12.21%
MS turnings	71.300	29,780.00	60.000	35,500.00	+5,720.00	+19.21%
Brass honey	9.800	4,92,000.00	0.000	—	—	—

Behaviour

- **Period** uses the same URL-held date control and FY presets as every other page. Default: This month.
- **"Check one material"** (POC) becomes a click on a row. That opens a sheet with the six POC boxes (bought, avg buy, sold, avg sell, difference, %) and the rate trend line from `series` (`bucket=week` for ranges ≤ 3 months, otherwise `month`), buy solid and sell dashed, labelled at the line ends.
- **Colour** is green/red only on per-material differences, always with a sign and a word for screen readers ("higher", "lower"). The blended cards are never coloured (D3).
- **Empty sides** show "—" with the tooltip "Nothing sold in this period", never ₹0.00.
- **Tooltip on "Difference"**: "Average selling rate minus average buying rate in this period. It is not profit per ton: stock bought earlier may have been sold now."
- VIEWER sees the page fully; it is read-only.

Why the overall figure is kept but downgraded

The owner used the POC's overall spread, so removing it would be a regression. The UI doc is right that mixing ₹4.9 lakh/t brass with ₹34,000/t HMS gives a number that moves with the product mix, not with pricing. Labelling it "blended", adding the tooltip and putting the per-material table first keeps the figure honest.

F9–F14: data layer

F9 · Complete error dictionary

The UI doc maps 11 situations. The backend can return **33** API error codes today, and this spec adds **4**. `lib/errors.ts` must cover every one, using `error.details` for the numbers. Raw codes are never shown; the `requestId` appears only behind "Copy reference" on 500s.

HTTP	CODE	WHAT THE USER READS (AND THE ONE-TAP FIX)
422	<code>VALIDATION_ERROR</code>	Message under each field from <code>details.issues[].path</code> , plus a focused "There is a problem" summary.
400	<code>INVALID_JSON</code>	"The app sent something the server could not read. Reload the page and try again." (Reload)
400	<code>INVALID_ID</code>	"This link is broken." (Go to list)
413	<code>PAYLOAD_TOO_LARGE</code>	"This entry is too long. Shorten the notes and save again."
401	<code>INVALID_CREDENTIALS</code>	"Email or password is wrong." (on the login form)
401	<code>TOKEN_EXPIRED</code>	Silent: refresh once, replay the request. Never shown.
401	<code>UNAUTHENTICATED</code> , <code>INVALID_REFRESH_TOKEN</code>	Login dialog over the form: "Your session has ended. Log in again to save. Your entries are kept."
403	<code>FORBIDDEN</code>	"You have view-only access. Ask the owner if you need to make changes."
403	<code>CSRF_CHECK_FAILED</code> new B16	"Your session could not be confirmed. Reload the page and log in again." (Reload)
404	<code>NOT_FOUND</code> , <code>COMPANY_NOT_FOUND</code> , <code>MATERIAL_NOT_FOUND</code> , <code>PO_NOT_FOUND</code> , <code>PURCHASE_NOT_FOUND</code> , <code>SALE_NOT_FOUND</code>	Full-page state: "We couldn't find this sales order. The link may be wrong." (Go to Sales Orders, or the matching list)
409	<code>DUPLICATE_VALUE</code>	On the field named by <code>details.fields</code> (map <code>normalizedName</code> →Name, <code>normalizedInvoiceNumber</code> →Invoice no., <code>normalizedChallanNumber</code> →Challan no., <code>normalizedPoNumber</code> →PO no., <code>gstNumber</code> →GSTIN): "Invoice no. ST/26-27/418 is already used by this supplier." (Open that record)
409	<code>COMPANY_TYPE_IN_USE</code>	"This company already has purchases (or orders), so it must stay a supplier (or buyer). Choose Both instead." (Set to Both)
409	<code>PO_QUANTITY_EXCEEDED</code>	"PO-0142 has only 11.750 t left to deliver. You entered 12.000 t." (Use 11.750 t)
409	<code>INSUFFICIENT_STOCK</code>	"Only 71.500 t of HMS 1 is in the yard on that date." (Use max)
409	<code>INSUFFICIENT_SOURCE_STOCK</code>	"Only 9.200 t of HMS 1 from Shree Ganesh Metals is left." (Use 9.200 t · Choose another supplier)
409	<code>NEGATIVE_STOCK_HISTORY</code>	"This change would make HMS 1 stock go below zero on 18/09/2026 (by 2.300 t). Change the quantity or date, or edit the later deliveries first." (View stock history)
409	<code>NEGATIVE_SOURCE_STOCK_HISTORY</code>	Same sentence, naming the supplier: "...HMS 1 from Shree Ganesh Metals would go below zero on 18/09/2026."
409	<code>PO_QTY_BELOW_SOLD</code>	"18.250 t is already delivered on this order. The order quantity cannot be lower." (Use 18.250 t)
422	<code>PO_DATE_AFTER_SALES</code>	"The first delivery on this order is dated 12/09/2026. The order date must be on or before it." (Use 12/09/2026)
422	<code>PO_NOT_AVAILABLE</code>	"This order is cancelled. Choose another order." (Change order)
422	<code>SALE_BEFORE_PO_DATE</code>	"The delivery date is before the order date (09/09/2026)." (Use 09/09/2026)

HTTP	CODE	WHAT THE USER READS (AND THE ONE-TAP FIX)
422	<code>INVALID_PURCHASE_COMPANY</code> , <code>INVALID_SALE_COMPANY</code> , <code>INVALID_SOURCE_COMPANY</code>	"Sharma Traders is inactive or is not set up as a supplier (buyer)." (Choose another · Edit company)
422	<code>INVALID_MATERIAL</code>	"This material is inactive. Reactivate it in Materials or choose another."
422	<code>EXPORT_TOO_LARGE</code> new B14	"That is more than 50,000 rows. Choose a shorter date range and export again."
409	<code>IDEMPOTENCY_IN_PROGRESS</code> new B17	Silent: wait <code>Retry-After</code> , repeat once with the same key. Shown only if it repeats: "Still saving... Check the list before trying again."
422	<code>IDEMPOTENCY_KEY_REUSED</code> new B17	Client bug: generate a new key, retry once silently, then treat it as a 500.
429	<code>TOO_MANY_AUTH_ATTEMPTS</code>	"Too many login attempts. Wait 15 minutes and try again." (The window is 15 min / 10 attempts.)
429	<code>TOO_MANY_REQUESTS</code>	Non-blocking: "Too many requests in a short time. Wait a few seconds and try again." (Try again)
500	<code>INTERNAL_ERROR</code>	"Something went wrong on our side and this purchase was not saved. Your entries are still here." (Try again · Copy reference)

Reconciliation codes (`PO_OVER_DELIVERED`, `SALE_AMOUNT_MISMATCH`, `NEGATIVE_MATERIAL_STOCK` and 6 more) only appear in `npm run reconcile` output, never in API responses, so the dictionary does not need them.

F10 · Query keys and what to refresh after each save

Saves wait for the server on stock and money (UI doc P0-9), then invalidate exactly these keys. Missing one leaves a stale number on screen; invalidating everything makes the app feel slow on weak data.

AFTER	INVALIDATE
Purchase create / edit	<code>purchases</code> , <code>dashboard</code> , <code>materials</code> (position), <code>source-stock</code> , <code>capacity</code> , <code>company-stats</code> , <code>activity</code> , <code>trend</code> , <code>rates</code> , <code>last-rate</code>
Sale create / edit	<code>sales</code> , <code>sales-pos</code> , <code>sales-po-summary</code> , <code>sales-po:{id}</code> , <code>dashboard</code> , <code>materials</code> , <code>source-stock</code> , <code>capacity</code> , <code>company-stats</code> , <code>activity</code> , <code>trend</code> , <code>rates</code>
Sales PO create / edit / cancel	<code>sales-pos</code> , <code>sales-po-summary</code> , <code>sales-po:{id}</code> , <code>dashboard</code> , <code>materials</code> , <code>capacity</code> , <code>company-stats</code> , <code>activity</code> ; plus <code>sales</code> when company or material changed (cascade)
Company create / edit	<code>companies</code> , <code>company:{id}</code> , <code>company-stats</code> ; names shown in lists refresh on their next fetch
Material create / edit	<code>materials</code> , <code>material:{id}</code> , <code>dashboard</code> (opening stock changes position)

Stale times: masters 5 min · lists 30 s · dashboard 60 s (the UI doc refetches every 60 s) · capacity 0 (always fresh) · activity 30 s.

F11–F14 · Smaller client items

F11 · URL filters with ids

Filters are stored as `?companyId=66f...&materialId=66f...`.
The chip shows the name from the cached master list ("Supplier: Shree Ganesh Metals ×"). An unknown id (deleted or deactivated) shows "Unknown supplier" and can still be cleared.
No backend slug field.

F12 · Export button

Calls B14 with the current URL query using `fetch` plus the Bearer header, receives a blob and saves it with the `Content-Disposition` filename. A plain link cannot carry the token. A spinner shows on the button; 422
`EXPORT_TOO_LARGE` goes through F9.

F13 · Purchase "last rate" default

When supplier and material are both set and the rate is still empty, the form fills it from B5 and shows the muted line "Last rate from Shree Ganesh: ₹34,500.00 on 27/09/2026". If `scope` is `MATERIAL`: "Last rate for HMS 1 (any supplier)". It never overwrites a typed rate.

F14 · Cookie session and idempotency

When the backend runs `AUTH_REFRESH_TRANSPORT=cookie`:
`credentials: 'include'` on auth calls, the
`X-Requested-With: metalix` header, and the refresh token is never stored. On every create: `Idempotency-Key` = a UUID made when the form opens. It is reused by "Try again", and a new one is made after success or when the user edits a field.

Test plan and definition of done

The existing 111 tests are the safety net and must pass **unchanged**. Every new behaviour gets integration tests against the real replica set, the same way the current suite works (`vitest` + `supertest` + `mongodb-memory-server`).

New test files

FILE	COVERS	≈ TESTS
<code>integration/sales-capacity.test.js</code>	B1: parity with create/edit for each binding limit, floor on backdated sales, <code>excludeSaleId</code> , blockers, route order, VIEWER	14
<code>integration/list-extras.test.js</code>	B2 summaries on 3 lists; B4 sort on 5 lists, invalid sort, stable paging	12
<code>integration/salesPO-summary.test.js</code>	B3 multi-status, <code>OPEN</code> alias, counts, filters	6
<code>integration/purchases-last-rate.test.js</code>	B5 exact, fallback, none, backdated ordering	4
<code>integration/search.test.js</code>	B6 every new searchable field, vehicle code-form fix, regex escaping	10
<code>integration/reports-extended.test.js</code>	B7 position = dashboard row; B8 totals/counts/parties; B9 company stats vs source-stock; B11 buckets, zero-fill, clipping, 400-bucket cap; B12 weighted averages, null difference	18
<code>integration/activity.test.js</code>	B10 ordering by <code>createdAt</code> , cursor paging with equal timestamps, <code>types</code> filter, PO status now	5
<code>integration/export.test.js</code>	B14 headers, BOM, rows = list total, filters, injection guard, size cap	6
<code>integration/auth-cookie.test.js</code>	B16 all three transport modes, rotation, CSRF header, logout clears cookie	8
<code>integration/idempotency.test.js</code>	B17 replay, reused key, concurrent, retry after failure, TTL index present	6
<code>unit/ledger-floor.test.js</code>	<code>ledgerFloor()</code> : flat, dip on a later day, same-day in/out, empty ledger	5
<code>unit/csv.test.js</code>	Escaping, quotes, newlines, injection prefixes	4

Plus updates to `database.test.js` (migration 002 indexes, idempotent re-run) and a seed smoke test that runs `seed` against the memory server and asserts the dashboard shows one "buy needed" material. Expected total: **111 existing + about 100 new**.

The one test that matters most

Capacity parity, randomised. Build a random ledger (purchases, POs and deliveries on random past dates across 3 suppliers). Then, for 50 random (PO, supplier, date) picks, call `/sales/capacity` and assert that a delivery of exactly `maxAllowedTons` is accepted and `maxAllowedTons + 0.001` is rejected. Roll back between picks. If this passes, the panel and the server can never disagree.

Frontend checks per item

- MSW handlers for every new endpoint, with fixtures copied from the response examples in this document.
- F1: typing above max shows the meter over 100% and the save button still works (the server decides); a 409 refetches capacity.
- F9: a unit test asserting every code in the backend list has a dictionary entry (the list is exported from a generated `error-codes.json`).
- F10: after each save type, assert the listed query keys were invalidated and no others.
- Accessibility per UI doc page 24: meters have text equivalents, sort headers use `aria-sort`, green/red differences also carry a sign and a word.

Definition of done (per phase)

1. `npm run check` green: lint plus all tests, old and new.
2. README: API table, new env vars and "deviations from spec" (D1–D6 answers) updated.
3. `npm run reconcile` on a copy of the Atlas data reports no new findings.
4. Phase report to the owner: what shipped, what changed for users, what is next.

Rollout, configuration and file changes

Deploy order

PHASE	STEPS	ROLLBACK
A	Deploy backend (no migration). Deploy frontend F1, F2, F9, F10, F13. The old frontend keeps working because every change is additive.	Redeploy the previous build.
B	<code>npm run migrate</code> (002: indexes only; it runs online on Atlas), then deploy backend, then frontend F3–F8.	Redeploy the previous build; extra indexes are harmless.
C	Deploy backend with B14/B15; the seed script is used on staging only.	Redeploy.
D	1. Deploy backend with <code>AUTH_REFRESH_TRANSPORT=both</code> . 2. Deploy the frontend with F14. 3. After <code>REFRESH_TOKEN_TTL_DAYS</code> , when every old session has rotated, switch to <code>cookie</code> . Idempotency needs no flag; the header is optional.	Set transport back to <code>body / both</code> .

New environment variables

Add each one to the Zod schema in `src/config/env.js` (with the defaults below) and to `.env.example`.

VARIABLE	DEFAULT	USED BY
<code>AUTH_REFRESH_TRANSPORT</code>	<code>body</code>	B16: <code>body</code> <code>cookie</code> <code>both</code>
<code>REFRESH_COOKIE_NAME</code>	<code>msm_rt</code>	B16
<code>COOKIE_SECURE</code>	<code>true</code>	B16; forced true when <code>NODE_ENV=production</code>
<code>COOKIE_SAMESITE</code>	<code>strict</code>	B16: <code>strict</code> <code>lax</code> <code>none</code> (none requires secure)
<code>COOKIE_DOMAIN</code>	<code>unset</code>	B16
<code>EXPORT_MAX_ROWS</code>	<code>50000</code>	B14
<code>IDEMPOTENCY_TTL_HOURS</code>	<code>24</code>	B17
<code>ALLOW_DB_RESET</code>	<code>false</code>	B15 <code>--reset</code> ; ignored when <code>NODE_ENV=production</code>

Files touched

New files

- `src/utils/search.js` (B6), `src/utils/csv.js` (B14)
- `src/middleware/idempotency.js`, `src/models/IdempotencyKey.js` (B17)
- `src/middleware/csrf.js` (B16)
- `src/db/migrations/002-activity-and-idempotency.js` (B13)
- `scripts/seed.js` (B15)
- 12 test files listed in chapter 9

Modified files

- **services:** stock (floor), sale (capacity), purchase / salesPO / company / material (summary, sort, search, include), report (helpers + 4 new reports), auth (cookie)
- **repositories:** base (`summarize`), salesPO (multi-status, summary facet, sort branch)
- **validations:** common (`sortQuery`), every list schema, sale (capacity), report (4 schemas)
- **routes/controllers:** sale, purchase, salesPO, report, company, material, auth
- **models:** Purchase, SalesPO, Sale (`createdAt` index)
- `app.js` (CORS headers/credentials), `config/env.js`, `migrations/index.js`, `package.json` (`seed` script, `cookie-parser`), `README.md`

New endpoints at a glance

	PATH	ITEM	ROLE
GET	/api/v1/sales/capacity	B1	OWNER, VIEWER
GET	/api/v1/sales-pos/summary	B3	OWNER, VIEWER
GET	/api/v1/purchases/last-rate	B5	OWNER, VIEWER
GET	/api/v1/reports/companies	B9	OWNER, VIEWER
GET	/api/v1/reports/activity	B10	OWNER, VIEWER
GET	/api/v1/reports/trend	B11	OWNER, VIEWER
GET	/api/v1/reports/rates	B12	OWNER, VIEWER
GET	/api/v1/{purchases sales-pos sales}/export.csv	B14	OWNER, VIEWER

Plus new optional parameters on existing endpoints: `sort` (all lists), multi-value `status` (sales-pos), `include=position` + `asOf` (materials), `include=stats` + `from / to` (companies), `include=parties` (dashboard), and the `Idempotency-Key` header on the five create routes.

Every POC feature, and where it lives now

A check that nothing from `metal-scrap-management-poc-v8` is lost. "Existing" means the current backend already does it; the UI/UX doc covers the screen unless an F-item is named.

POC FEATURE	STATUS	WHERE
Global from/to date filter on every view	Existing	<code>from / to</code> on all lists and reports; URL-held (F11)
Dashboard KPI cards (6)	Added	4 cards + "More figures" strip · B8, F7, D5
View one material (cards)	Added	F7 via <code>dashboard?materialId</code>
Material-wise business position table (with supplier / PO-customer names)	Added	B8 <code>include=parties</code> , F7
Company-wise purchases / sales panels	Added	B9, F7
Pending POs with material position	Added	B3 + B4 + B7, F4, F7 "Orders waiting"
Recent activity (purchase, sale, PO)	Added	B10, F7
Companies: create, edit, type, GST, address, duplicate name check	Existing	<code>/companies</code> (plus per-GST uniqueness)
Company search by name / contact / GST, type filter	Added	B6 (contact, address), existing type filter
Companies table: purchases, stock used, stock available, POs, sales	Added	B9 <code>include=stats</code> , F6
Company overview panel	Existing	<code>/reports/companies/:id</code> , UI company detail
"Stock used from this purchase company" table	Existing API	<code>/sales?sourceCompanyId</code> ; new tab F6
Materials: create, edit, notes, duplicate name check	Existing	<code>/materials</code>
Material unit (Tons / Kg / Pieces)	Dropped	Decision D4: tons only, kg typed and converted in the input
Material cards with full position	Added	B7, F5
Rate analysis page (overall + per material, difference ₹ / %)	Added	B12, F8, D3
Purchases: create, edit, auto total, vehicle, invoice, notes	Existing	<code>/purchases</code> (amount computed server-side)
Purchase search by material / supplier / invoice / vehicle	Added	B6
"Current supplier stock" column	Added	F3 via <code>/reports/source-stock</code>
Sales POs: create, edit, qty ≥ delivered, unique PO no., cancel	Existing	<code>/sales-pos</code> (unique per customer, D2)
PO status (Pending / Partly / Completed / Cancelled) and filter	Existing + added	Derived <code>displayStatus</code> ; multi-status B3
"Purchase balance" column on POs	Added	F4 via B7
PO edit cascades customer / material to deliveries	Existing	<code>salesPOService.update</code> with ledger re-checks
PO rate edit reprices old deliveries	Changed	Decision D1: rate snapshot kept (<code>poRateAtSale</code>)
Deliveries: PO locked rate, customer from PO, vehicle, challan	Existing	<code>/sales</code>
Customer → only POs with quantity left	Added	B3 <code>status=OPEN</code> + <code>companyId</code> , F1
"Stock from" list with available tons per supplier	Added	B1 <code>sources</code> , F1

POC FEATURE	STATUS	WHERE
Max delivery = min(PO left, supplier stock)	Improved	B1 also includes the yard limit and later-day floors (the server already enforced them)
Editing a delivery does not count itself	Existing + exposed	<code>excludeSaleId</code> in B1
Sale search by customer / supplier / PO / material / challan / vehicle	Added	B6
Reset demo data	Replaced	B15 <code>npm run seed -- --demo --reset</code> (never in production)
Browser localStorage storage, float maths	Replaced	MongoDB Atlas, Decimal128, transactions, ledger replay (existing)

UI/UX doc page 30 "backend additions": all covered

UI DOC REQUEST	ITEM
<code>GET /sales/capacity?poId&sourceCompanyId&saleDate&excludeSaleId</code>	B1
Last rate for a supplier + material	B5
<code>GET /reports/rates</code> (day / week / month buckets)	B12 (+ B11 for the volume trend)
Refresh token in an httpOnly cookie	B16
Idempotency key on create endpoints	B17